AgriVision: Leaf-Photograph Crop Disease Diagnosis with Treatment Advice, and a Measured Study of What Works on Field Photographs

Pishke Vishal (23K91A05J2)
Dept. of Computer Science and Engineering
[College name]
[City], India
[email address]

Kyatham Nithisha (23K91A05D1)
Dept. of Computer Science and Engineering
[College name]
[City], India
[email address]

Mekala Tharun Kuruma (23K91A05F1)
Dept. of Computer Science and Engineering
[College name]
[City], India
[email address]

Nakka Sreenith (23K91A05H0)
Dept. of Computer Science and Engineering
[College name]
[City], India
[email address]

V Krishna Teja (guide)
[DESIGNATION], Dept. of Computer Science and Engineering
[College name]
[City], India
[email address]

***Abstract*---**Crop diseases cost farmers a large share of their harvest, and an expert is rarely at the field in time. This paper presents AgriVision, a web application in which a farmer uploads a leaf photograph, names the crop and receives the most likely disease, its severity and chemical, organic and cultural treatment advice for 38 classes, with the interface in English, Hindi and Marathi. The application is measured, not only described. On the PlantDoc field-photograph benchmark, with 2,342 training and 236 test photos, the colour and texture heuristic that the shipped application uses names the correct class for 9.1 % of photos when the crop is chosen, and fails outright on 32.2 %. The compact CNN of the training script reaches 12.0 %; a MobileNetV2 with ImageNet weights reaches 50.3 % accuracy and macro-F1 0.49 over three seeds, but is not yet installed in the application. We report these results, seven defects found by testing and code review, and the parts of the original plan (fertiliser, yield and weather advice) that are not implemented.

***Index Terms*---**computer vision, crop disease detection, leaf images, MobileNetV2, PlantDoc, transfer learning, treatment advice.

I. INTRODUCTION

Plant diseases are estimated to cost the world about USD 220 billion a year [1], [2]. A farmer who sees spots on a leaf needs to know quickly which disease it is and what to spray, but extension officers and plant pathologists are few and far from the field, so diagnosis is delayed and treatment is guessed. Convolutional networks classify leaf images with accuracies above 99 % on laboratory collections such as PlantVillage [2], [3], [4], and compact networks suitable for ordinary computers exist [2]. Field photographs are harder: the PlantDoc authors show that models trained on laboratory images fail on photographs taken in the open [5], and Barbedo lists background, symptom variation and small datasets as the main obstacles [6].

Most published work stops at the class label. A farmer also needs the severity, a treatment and prevention advice in a language they read. The base paper of this project, Foliagen [1], addresses a different problem, the scarcity of annotated foliage images, by generating them from single-leaf photographs; it does not return advice to a farmer. The gap this project works on is the join between a diagnosis and an actionable, multilingual treatment recommendation.

The contributions are: (1) a complete, deployable web application (React interface, FastAPI service over the original Flask product, SQLite store) that diagnoses a leaf photograph, returns severity and three kinds of treatment from a 38-class knowledge base, keeps a history with CSV and PDF export and offers an English, Hindi and Marathi interface; (2) a seeded benchmark on PlantDoc that measures the engine the application actually ships, the compact CNN of its training script, a transfer-learning reference and classical baselines under one protocol; (3) the finding that the shipped heuristic does not diagnose (about chance) while transfer learning reaches about 50 %; and (4) a list of seven defects found by tests and code review, with an honest account of what is not built. Section II surveys related work, Section III describes the method, Section IV gives the results and Section V concludes.

A. System Hardware

All measurements ran on one desktop computer: Intel Core i5-12500 (6 cores, 12 threads), 15.7 GB RAM, Windows 11 Pro, CPU only (no GPU). Other jobs shared the processor during training, so training times are indicative.

B. Software Requirements

Python 3.11.9, TensorFlow 2.21.0 (Keras 3), scikit-learn 1.5.0, OpenCV 4.11, Pillow 12.3, FastAPI, Flask 3.1, reportlab 4.1, Node 24.19, React 18.3, Vite 8, TypeScript 5.9, Tailwind CSS 4. Networks were built with TensorFlow [7] and classical models with scikit-learn [8].

II. LITERATURE SURVEY

Base paper. Pakka et al. [1] observe that leaf classifiers work on single leaves while real use needs in-field foliage images, and that annotated foliage datasets are scarce. Foliagen generates them without a generative network: single-leaf photographs (10,722 soybean images of seven diseases plus 132 more, and PlantVillage tomato images) lose their background with an automatic remover, are arranged by a spiral phyllotactic generator that follows plant structure, and are placed on field backgrounds with a chosen share of diseased leaves (1 to 20 %, about 15 % for the natural soybean set). About 1,000 images per disease category are generated for soybean and tomato. VGG19, ResNet50, DenseNet121, Swin Transformer and CBAM-ConvNeXt are trained on two RTX 6000 Ada GPUs (Adam,

batch 8, learning rate 1e-4, early stopping). Adding 5 % of a natural foliage set to generated data lets classifiers reach up to 98 % accuracy on the natural set, and pre-training on generated images followed by fine-tuning with few real ones also works. The authors note the approach depends on leaf-image quality and may need crop-specific structure for other crops. It covers two crops, produces data rather than a diagnosis, and has no recommendation step. This project does not implement Foliagen; it keeps its lesson that laboratory images transfer poorly to the field.

Related work. Datasets: PlantVillage [3] holds laboratory images and is the basis of most reports; PlantDoc [5] offers 2,598 web-scraped field photographs of 13 plants in 27 classes, with a 2,342/236 split, and TomatoVillage [9] adds real-world tomato images. CNN classifiers: Mohanty et al. [4] and Ferentinos [10] report 99 % or more on PlantVillage-style data; PD36-C [2] is an 18-layer network with 1,250,694 parameters and 99.53 % test accuracy on 87 thousand laboratory-style images, and Kanakala and Ningappa [11] report 96.4 % validation accuracy for a CNN on the same kind of collection. On PlantDoc the benchmark paper [5] reaches 44.5 to 70.5 % on cropped leaves and 13.7 to 29.7 % on raw photographs. Efficient backbones such as MobileNetV2 [12] built on depthwise separable convolutions [13] make training on a CPU feasible, and fine-tuning ImageNet weights [14] is the usual remedy for small data.

Position of this work. The surveyed classifiers stop at a label, and the one that offers a desktop application [2] works offline on laboratory-style images. Field studies stress that the test data must differ from the training data in the way real use differs [5], [6]. We therefore use PlantDoc, which is small and noisy but honest about field conditions, and we build the missing layer: severity and treatment advice, history and a three-language interface in one application, then measure how well the diagnosis inside it works.

TABLE I
RELATED WORK: APPROACH, DATA, REPORTED RESULT, LIMITATION

Work	Approach	Data	Reported result	Limitation
Pakka [1]	Generated foliage images	Soybean, tomato	Up to 98 % with 5 % real	Data only, 2 crops
Sherifi [2]	18-layer CNN, 1.25 M params	87 k lab images	99.53 % test	Lab data, no advice
Kanakala [11]	CNN and LSTM	87.9 k images, 38 cl.	96.4 % val. (CNN)	Random split
Singh [5]	VGG16, Inception on PlantDoc	2,598 field photos	13.7-70.5 %	Small, hard
Mohanty [4]	AlexNet, GoogLeNet	PlantVillage	99.35 %	Lab backgrounds
Ferentinos [10]	5 CNNs	87.8 k images, 58 cl.	99.53 %	Mostly lab
Barbedo [6]	Factor analysis	Public sets	Accuracy falls off-lab	No application

Table I shows the gap: reported accuracies are high on laboratory data and low on field photographs, and none of the surveyed works returns treatment advice in several languages through an application. The figures come from different data and protocols and are not ranked here.

III. METHODOLOGY

Architecture. Fig. 1 shows the three tiers. A React 18 interface with 12 page components (landing, sign-in, dashboard, detect, result, history, analytics, profile, disease guide) talks to a FastAPI service with 16 routes. The service imports the original Flask product unchanged: its inference module, its 38-class knowledge base and its database initialiser. One SQLite file holds two tables, users and results, and uploaded photographs are stored as files with random names. Sign-in returns an HMAC-signed bearer token valid for seven days; passwords are stored as salted hashes.

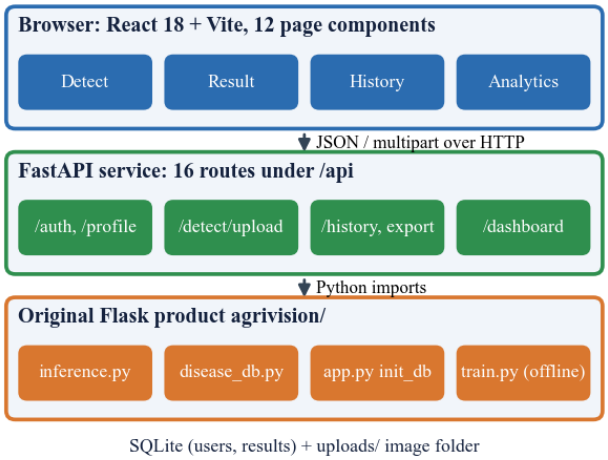


Fig. 1. System architecture: browser, FastAPI service and the original Flask modules, with the SQLite store and the image folder (full diagram: docs/diagrams/01).

Main flow. Fig. 2 follows one scan. The service refuses a request without a valid token, an extension other than JPG, PNG or WEBP, an empty file or a file above 10 MB. The inference module decodes the photograph, ranks the candidate classes, and the service attaches the knowledge card of the top class, stores the image and one row, and returns the top three classes with severity and treatments. The result screen, history list, CSV or PDF export and the analytics charts read the same rows.

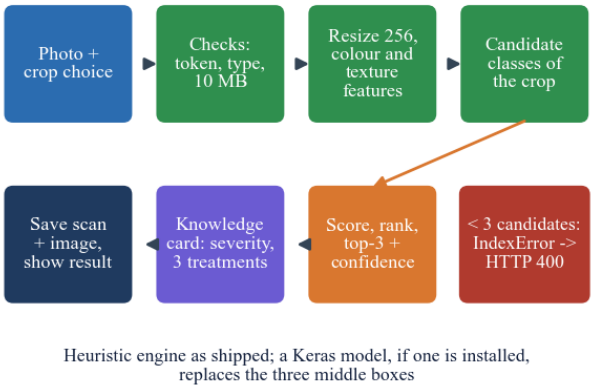


Fig. 2. Main flow of one diagnosis in the shipped application, including the failure for crop options with fewer than three candidate classes.

Heuristic engine. The engine that the application ships uses no trained network. The photograph is resized to 256 × 256 and five pixel shares are computed: green g (green above red by 0.03 and above blue by 0.01), brown b, yellow y, dark d (mean below 0.25) and white w (minimum above 0.72), plus the standard deviation σ of all values. The candidate classes are those of the chosen crop (all 38 if the crop is 'general'). A healthy class scores

$$s_h = 3g - 4(b + y) - 1.5d + 0.25 - |\sigma - 0.18|, \qquad (1)$$

a diseased class scores

$$s_d = 3.2b + 2.4y + 1.2d + 1.6w + 1.5 \cdot \max(0,\ 0.5 - g) + \text{bonus}, \qquad (2)$$

where the bonus adds a multiple of white, brown or yellow share for names containing mildew, rust, blight, mosaic or mite, and a jitter of up to 0.35 derived from a hash of the file makes the order stable per image. The top class receives the displayed confidence

$$c = \min(0.99,\ \max(0.62,\ 0.62 + 0.22 \cdot \min(1.6,\ s))), \qquad (3)$$

so the confidence never falls below 0.62; the second and third scores are small fractions of c taken from the same hash. A crop with fewer than three candidate classes (10 of 19 options) leaves the ranking short and raises an error, which the API reports as an unreadable image.

Design rationale. The heuristic was kept as the shipped engine because it needs no model file and answers in milliseconds on a laptop, which suited a first deployable version, and the knowledge base is kept apart from the engine so that a better classifier can be installed without touching the advice. The compact network follows the depthwise-separable design of mobile models [13] to stay small; MobileNetV2 is added as a reference because ImageNet pre-training [14] is the standard answer to a training set of about two thousand photographs. No cropping or leaf detection is applied before classification, so the backgrounds of the web photographs reach the network.

Knowledge base. The 38 classes (14 crops, 12 healthy) carry a severity (12 high, 14 medium, 12 none), symptoms, life cycle, prevention and three treatments (chemical, organic, cultural) with cost text in rupees; 26 diseases have curated text and the remaining ones use generic text for their kind (fungal, bacterial, viral, pest). The interface strings, but not this text, are translated into Hindi and Marathi.

Data and protocol. PlantDoc [5] (licence CC-BY-4.0) in the local folder layout has 2,342 training photographs in 28 classes (2 to 180 per class) and 236 test photographs in 27 classes. Photographs are resized to 224 × 224. A stratified 10 % of the training folder is held out for early stopping, so the test photographs never influence training or selection; the shipped script instead validates on the test folder. Every learned model is run with seeds 42, 43 and 44 and reported as mean ± standard deviation.

Networks. The compact CNN of the training script has five separable-convolution blocks [13] of 32, 64, 128, 256 and 256 filters, each followed by batch normalisation [15], 2 × 2 max-pooling and dropout 0.15, then global average pooling, dropout 0.3, a dense layer of 256 units and a softmax over 28 classes: 189,655 parameters, with random flip, rotation, zoom and contrast in the model. It is trained with Adam [16] at 1e-3, batch 32, up to 60 epochs, early stopping on validation accuracy and learning-rate halving on plateau. The reference model is MobileNetV2 [12] with ImageNet weights: a new softmax head is trained with the backbone frozen, then the top 30 layers are fine-tuned at 1e-4 (2,293,852 parameters). The loss is

$$L = -(1/N) \sum_i \log p_i(y_i). \qquad (4)$$

Two classical baselines use 43 hand-made features (HSV histograms, grey-level and gradient statistics): a random forest [17] with 300 trees and a logistic regression.

Metrics. Top-1 accuracy is the share of photographs whose highest-ranked class is the labelled one; top-3 accuracy counts the label among the three displayed classes. For each class c, precision P and recall R give the F1 score, and macro-F1 averages it over the 27 classes present in the test folder, so every class counts equally however few photographs it has:

$$F1_c = 2\,P_c\,R_c / (P_c + R_c), \quad \text{macro-}F1 = (1/27) \sum_c F1_c. \qquad (5)$$

Two derived scores check whether the engine at least separates healthy from diseased leaves and finds the right crop: healthy-versus-diseased accuracy, from the class names, and crop accuracy. Latency is the time from sending the upload to receiving the reply on the loopback address.

Implementation and error handling. The interface keeps the token in the browser, attaches it to every call and signs the user out on a 401 reply. Every error from the service has one JSON shape with an error code, a message and a detail string, so the interface can show the message to the farmer. A result belongs to one user: another user's request for it returns 404, deleting a result removes the row and the image file, and a single scan can be exported as CSV or PDF. Photographs are saved under random names, so the image address itself acts as the capability to view it. The launcher fixes the hash seed, which keeps the heuristic's jitter identical from run to run.

Verification. An end-to-end test of the API passes 39 checks. A second script adds 51 unit and API checks (token

forgery and expiry, upload limits, injection-style input, paging, cross-origin requests): 46 pass and 5 reproduce defects of the product, listed in Section IV. A browser check script covers layout at three widths, language and theme switching; it passed 55 checks in the earlier frontend session and was not re-run for this paper.

IV. RESULTS

Test conditions. All accuracies are on the 236 official PlantDoc test photographs (27 classes present), never used for fitting. Macro-F1 averages the 27 class F1 values. The heuristic is run with five hash seeds (seed 0 is what the launcher sets), learned models with three training seeds. Chance is 2.6 % on 38 classes and 36 % if the crop is known; always answering the most frequent class gives 5.1 %.

TABLE II
TEST RESULTS ON PLANTDOC (236 PHOTOS), PERCENT UNLESS STATED

Engine	Top-1	Macro-F1	Top-3	Healthy / diseased
Heuristic, crop chosen	9.1	0.090	34.9	35.1
Heuristic, crop 'general'	1.3	0.014	4.2	33.1
Logistic regression	13.1	0.120	28.4	80.1
Random forest	21.6	0.199	36.0	79.7
Compact CNN (3 seeds)	12.0 ± 0.5	0.086	30.9	76.8
MobileNetV2, head only	48.7 ± 1.4	0.467	-	-
MobileNetV2, fine-tuned	50.3 ± 0.9	0.490	79.8	88.8

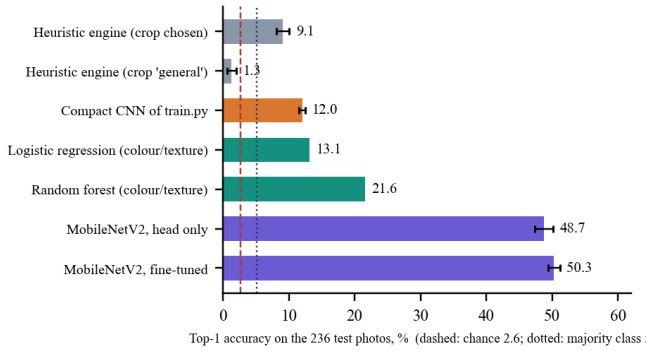


Fig. 3. Top-1 accuracy of every engine measured on the PlantDoc test photographs (error bars: standard deviation over seeds).

The shipped heuristic. With the crop chosen it is correct on 9.1 ± 1.0% of photographs (10.6 % with hash seed 0) and has the right answer among its top three on 34.9%, which is close to the 36 % expected by chance within a crop; with the crop left on 'general' it reaches 1.3 ± 0.7%. It cannot tell healthy from diseased: only 10.2% of healthy leaves are called healthy, and its healthy-versus-diseased accuracy (35.1%) is below the 61.9 % obtained by always saying 'diseased'. Its displayed confidence averages 95 % while top-1 is right on only 21 %, 22 % and 14 % of the photographs in the three confidence bins, and the three displayed scores add up to more than 102 % for 82 % of them (up to 112 %). Fig. 4 plots this and Fig. 5 shows a late-blight leaf called healthy with 97.2 %.

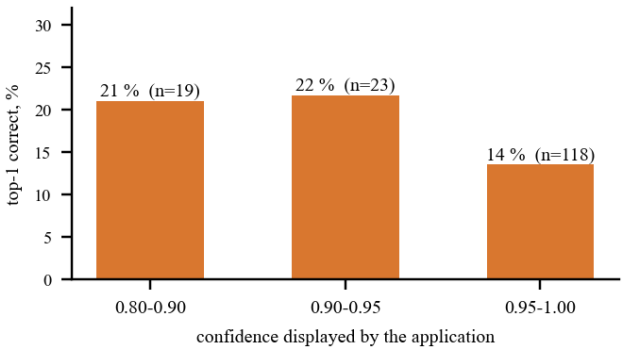


Fig. 4. Correctness of the heuristic engine by the confidence it displays (160 answerable test photographs): higher confidence is not more often right.



Fig. 5. Output screen (result): a PlantDoc tomato late-blight leaf diagnosed 'Tomato - healthy', 97.20 % by the heuristic engine; the note on the screen says the score is not a calibrated probability.

Failure by design. For 32.2 % of the test photographs the application gives no answer: the crop options blueberry, cherry, orange, peach, pepper, raspberry, soybean, squash and strawberry have fewer than three candidate classes, the ranking is too short, and the API replies HTTP 400 'could not be read as an image' to a valid photograph (Fig. 6). On the answered photographs the accuracy is 13.4 ± 1.5%.

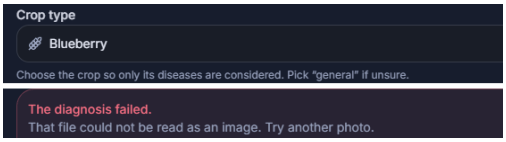


Fig. 6. Output screen (detect): crop 'Blueberry' with a valid leaf photograph; the application answers 'The diagnosis failed. That file could not be read as an image.'

Learned models. The compact CNN reaches 12.0 ± 0.5% (macro-F1 0.086) and under-fits: its training accuracy is 22.2 ± 2.8%. Validating on the test folder as the shipped script does gives 13.6 % and removing the learning-rate decay 12.7 %, so neither the selection leak nor the schedule explains the low value; with 2,342 photographs the small

network does not learn from scratch. A random forest on hand-made colour and texture features is better (21.6 %), and MobileNetV2 with ImageNet weights is far better: head-only 48.7 ± 1.4%, fine-tuned 50.3 ± 0.9%, top-3 79.8%, healthy-versus-diseased 88.8%, at 723 s of CPU training and a median of 89 ms per photograph. The best classes are strawberry (F1 1.00), grape black rot (0.93), grape (0.92) and corn rust (0.89); bell-pepper leaf spot, tomato bacterial spot and tomato mosaic virus are never recognised, visually similar diseases of the same crop being confused.

Per class. Fig. 7 shows that the fine-tuned network does best on classes with a distinctive look (strawberry, grape, raspberry, corn rust) and worst on the three tomato and pepper spot and virus classes whose symptoms resemble one another. The compact network recognises only a few classes at all, mostly tomato yellow leaf curl virus (F1 0.67 for seed 42) and strawberry (0.38). With 4 to 12 test photographs per class these per-class values are indicative only.

Comparison with the base papers. Foliagen's up to 98 % [1] measures classifiers on soybean and tomato foliage with generated training data and GPUs; PD36-C's 99.53 % [2] is on laboratory-style images of 38 classes. Neither is comparable with the numbers above, which are for field photographs, 28 classes, 2,342 training images and a CPU. The nearest like-for-like figure is the PlantDoc paper's 13.7 to 29.7 % for raw photographs [5], which the fine-tuned MobileNetV2 exceeds, while the shipped heuristic and compact CNN do not.

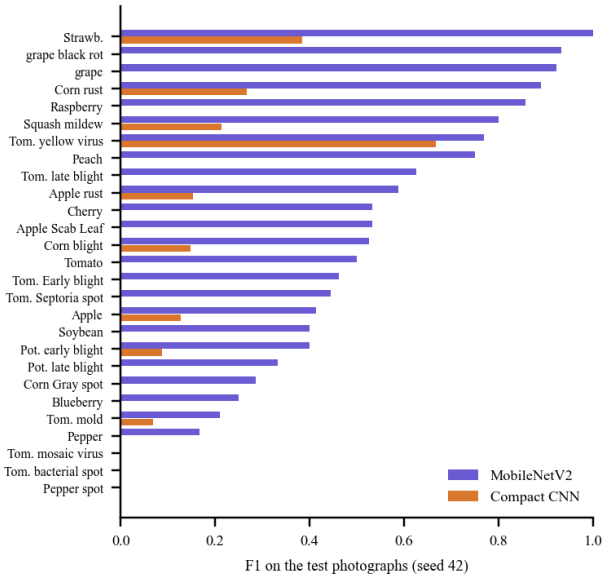


Fig. 7. Per-class F1 of MobileNetV2 and of the compact CNN on the 27 test classes (seed 42); several classes score zero.

Application and tests. On the local machine one diagnosis takes a median of 23 ms from upload to reply (95th percentile 82 ms, 100 photographs) and the history and dashboard calls about 10 ms; no concurrent-load test was made. Testing and code review found seven defects (Table III); the first is the crop failure above, and the others are listed with their evidence.

TABLE III
DEFECTS FOUND BY TESTS AND CODE REVIEW

ID	Defect	Evidence
D1	Crop options with fewer than 3 candidate classes raise an error (HTTP 400)	10 of 19 crop keys; 32.2 % of test photos
D2	Three displayed scores add up to more than 100 %; confidence never below 0.62	82 % of 160 photos above 102 %, max 112 %
D3	Default token-signing secret is public	A token forged with it is accepted
D4	No rate limit on sign-in	8 wrong passwords in 2.8 s all answered
D5	Training script validates on the test folder and mislabels the two grape classes	27 test class names against a 28-class model
D6	Training script docstring claims about 1.1 M parameters	Measured 189,655
D7	Reported latency includes a constant 40 ms	Server median 47 ms against 23 ms measured by the client

Cost. The compact CNN trains in 478 s on the CPU and answers in a median of 70 ms per photograph; MobileNetV2 needs 723 s and 89 ms. The saved compact network is 2.4 MB. The heuristic costs about 20 ms in the process and needs no training, which is its only advantage. Fine-tuning gains only about 1.6 points over the head alone, so on this small set most of the benefit comes from the pre-trained features.

Threats to validity. The test folder has 4 to 12 photographs per class, so single-class F1 values are noisy and a difference of one photograph changes them by 8 to 25 points. PlantDoc images are scraped from the web, and whether the same photograph appears in both folders was not checked. Hash seeds and training seeds vary only the sources of randomness we control; the split itself is fixed. The dosages in the knowledge base are curated text, not region-specific recommendations, and were not compared with local agronomic guidance.

Objectives against outcomes. (1) A labelled dataset was assembled from PlantDoc, not collected by the team. (2) Resizing and in-model augmentation were done; the shipped engine only resizes. (3) A CNN was trained, but it is inaccurate and not installed. (4) Treatment advice exists for all 38 classes; fertiliser advice does not. (5) The web interface exists with an interface-only translation. (6) Per-class accuracy was measured; the advice was not validated. The study is limited to one dataset of 236 test photographs, three seeds without a significance test, no field trial or user study, and no agronomist review.

V. CONCLUSION AND FUTURE SCOPE

AgriVision works end to end as a web application: sign-in, photograph upload, a top-3 diagnosis with severity, treatment and prevention advice for 38 classes, history, exports, analytics and an English, Hindi and Marathi interface, with 39 API checks passing. Against the objectives, the dataset, preprocessing, a CNN, the disease-to-treatment mapping, the web interface and per-class evaluation were done; the diagnosis engine in the application is not accurate, multilingual support covers only the interface, fertiliser, yield and weather advice are not built, and the advice has not been checked by an agronomist. The key finding is that field photographs need transfer learning: MobileNetV2 reaches 50.3 % where the heuristic reaches 9.1

% and the compact CNN 12.0 %, and that confidence shown by the application cannot be trusted.

What this means for a farmer today: the treatment text for a class is usable reference material, but the class name the application proposes should be confirmed with an agricultural officer before spraying, as the on-screen note already says. The measured gap between the heuristic and the pre-trained network is large enough that replacing the engine is the first step, and the benchmark and tests built here show when that step has worked.

Future work: install the fine-tuned network in the application and calibrate its probabilities; fix the crop-option failure and the training script; crop leaves or detect them before classification as the PlantDoc authors do [5]; add Grad-CAM explanations [18]; translate the treatment text; add fertiliser and weather advice; and test with farmers and with an agronomist.

REFERENCES

[1] N. Pakka, T. Wilkerson, L. Chen and N.-F. Tzeng, "Foliagen: Framework for crop foliage image generation from individual leaf images," IEEE Access, 2026, doi: 10.1109/ACCESS.2026.3727421.

[2] S. Sherifi, E. Rustemi, S. Ismaili, E. Rufati and F. Idrizi, "A compact and efficient 1.251 million parameter machine learning CNN model PD36-C for plant disease detection: A case study," arXiv:2604.11332, Apr. 2026.

[3] D. P. Hughes and M. Salathe, "An open access repository of images on plant health to enable the development of mobile disease diagnostics," arXiv:1511.08060, 2015.

[4] S. P. Mohanty, D. P. Hughes and M. Salathe, "Using deep learning for image-based plant disease detection," Frontiers in Plant Science, vol. 7, Art. no. 1419, 2016.

[5] D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat and N. Batra, "PlantDoc: A dataset for visual plant disease detection," in Proc. 7th ACM IKDD CoDS and 25th COMAD, Hyderabad, India, 2020, pp. 249-253; arXiv:1911.10317.

[6] J. G. A. Barbedo, "Factors influencing the use of deep learning for plant disease recognition," Biosystems Engineering, vol. 172, pp. 84-91, 2018.

[7] M. Abadi et al., "TensorFlow: A system for large-scale machine learning," in Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI), 2016, pp. 265-283.

[8] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[9] M. Gehlot, R. K. Saxena and G. C. Gandhi, "Tomatovillage: A dataset for end-to-end tomato disease detection in a real-world environment," Multimedia Systems, vol. 29, no. 6, pp. 3305-3328, 2023.

[10] K. P. Ferentinos, "Deep learning models for plant disease detection and diagnosis," Computers and Electronics in Agriculture, vol. 145, pp. 311-318, 2018.

[11] S. Kanakala and S. Ningappa, "Detection and classification of diseases in multi-crop leaves using LSTM and CNN models," J. Innovative Image Processing, vol. 7, no. 1, pp. 161-181, Mar. 2025, doi: 10.36548/jiip.2025.1.008.

[12] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in Proc. IEEE/CVF Conf. Comput. Vision Pattern Recognit. (CVPR), 2018, pp. 4510-4520.

[13] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in Proc. IEEE Conf. Comput. Vision Pattern Recognit. (CVPR), 2017, pp. 1251-1258.

[14] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in Proc. IEEE Conf. Comput. Vision Pattern Recognit. (CVPR), 2009, pp. 248-255.

[15] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in Proc. 32nd Int. Conf. Machine Learning (ICML), 2015, pp. 448-456.

[16] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. 3rd Int. Conf. Learning Representations (ICLR), 2015; arXiv:1412.6980.

[17] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001.

[18] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in Proc. IEEE Int. Conf. Comput. Vision (ICCV), 2017, pp. 618-626.